Day 3

System and Temperature

Two interview favorites. Two analogies:

- **Saying "I love you" to different people** → the response depends on your **relationship** (girlfriend = sweet reply; manager = HR complaint). → **System Role**
- **Asking "how do I score well in exams?"** → the answer depends on the person's **personality** (a straight friend says "study"; a reckless one says "kidnap the teacher"). → **Temperature**

1. System Role — defines the LLM's role & relationship to you

You send a `system` message **before** the user message. It tells the LLM *"who you are and what your relationship to me is"* — and it holds for the whole chat until another instruction overrides it.

Real-world use: A company buys Claude and builds multiple AI agents — like different employees in a company:

- **Developer** → writes and tests code.
- **Architect** → design, security, maintainability, overall system behavior.
- **Manager** → manages people.

Using the system role, one LLM can act as any of these:

- `"You are a senior architect"` → answers about security, reliability, partition tolerance, overall design.
- `"You are a junior developer"` → answers at code level (e.g., "an array index -1 was accessed → segfault"), won't flag high-level security issues.

So one subscription becomes an entire team: Claude #1 = architect, #2 = junior dev, #3 = tester.

2. Temperature — controls creativity / randomness

Range: [0,2] 0 to 2. (Passing 3 throws an error.)

Key idea: an LLM is a **prediction machine**, not a truth-telling machine. (Google retrieves facts; an LLM predicts the next likely token.)

Toddler analogy: a child who only knows the days of the week, told "Sunday, Monday," predicts "Tuesday" — not reasoning, just predicting from what she knows.

- **t = 0** (default) → the safest, most obvious prediction. Least creative.
- **higher t** → more creative / random, moves away from the obvious answer.

Food-delivery app naming example:

Temperature	Answer	Why
0	“Khana Khazana”	Safe, directly food-related
1	“Zomato”	Za + tomato — loosely related, more creative
2	“Swiggy”	No real meaning, just sounds nice

Match temperature to the use case:

- **Doctor AI agent** → low temp (~0). Needs facts, no creativity. Listen to symptoms → diagnose.
- **Storyteller AI** → high temp. Needs creativity/randomness (horror stories, quirky brand names).

3. Recap Table

	System Role	Temperature
Controls	The LLM’s role & relationship to you	The LLM’s personality / creativity
The question	Who are you to me?	How random/creative should the answer be?
Use case	Multiple agents in one org, distinct jobs	Different creativity for different tasks

4. Day 3 — Full Code (`system_temperature.py`)

Copy the Day 1 code, then add two things: a **system message** and the **temperature** parameter.

```
import os
from dotenv import load_dotenv
from groq import Groq

load_dotenv()
my_api_key = os.getenv("GROQ_API_KEY")
if not my_api_key:
    raise ValueError("API KEY not found")
```

```
client = Groq(api_key=my_api_key)
model = "llama-3.3-70b-versatile"

# --- SYSTEM ROLE ---
message_system = {
    "role": "system",
    "content": "You are my strict office colleague who also happens to be my
manager"
}

message_user = {
    "role": "user",
    "content": "I love you baby"
}

# System message ALWAYS goes first — order matters!
messages = [message_system, message_user]

# --- TEMPERATURE ---
response = client.chat.completions.create(
    model=model,
    messages=messages,
    temperature=1          # 0 = safe, 2 = max creative (3 = error)
)

print(response.choices[0].message.content)
```

Run (Mac):

```
uv run system_temperature.py     # 🍎 recommended
# or: python3 system_temperature.py
```

Experiments to try:

- `"You are my loving girlfriend"` + `"I love you baby"` → sweet reply.
- `"strict office colleague / manager"` → shocked, HR-policy warning.
- `"You are a brand manager; suggest ONE name for my food brand"`, then vary temperature → `0` : Savour, `1` : Tastio, `2` : Tastiore. Switch to a clothing brand → `0` : Vastra, `2` : Vesto.

Tip: Always put the `system` message **first** in the list. Otherwise “I love you” is processed before “you are my manager,” and the context breaks.

🍎 Windows → Mac Cheat-Sheet

Task	Windows (video)	🍎 Mac (for you)
Open terminal	`Windows + R` → cmd/PowerShell	`Cmd + Space` → "Terminal"
Activate venv	`.venv\Scripts\activate`	`source .venv/bin/activate`
Run Python	`python file.py`	`python3 file.py` or `uv run file.py`
Python version	`python --version`	`python3 --version`
Package manager	pip	pip3 (or `uv` — best)
Copy a file	right-click → paste	`cp source.env day1/` or Finder drag
Save in editor	`Ctrl + S`	`Cmd + S`
Path separator	`\` (backslash)	`/` (forward slash)
Install `code` cmd	bundled	VS Code → `Cmd+Shift+P` → "Install 'code' command"

🍎 **Biggest Mac tip:** Use `uv run file.py` throughout. It auto-uses the venv (no manual activate) and kills the `python` vs `python3` confusion. Just remember `uv run` for the whole course.

That's the complete series so far. The through-line: **Day 1** = get set up, **Day 2** = the mechanics of an LLM call (key → client → model → messages → response), **Day 3** = controlling that call (system = *who you are*, temperature = *how you answer*). Only the commands and shortcuts changed for Mac — the concepts are identical